DocuMind — Interview Answers

1. Explain it in one minute

> It's a document question-answering app I built end to end. The user drops in a PDF, a Word doc, a markdown file, or pastes a URL, and then asks questions in plain language and gets an answer with citations back to the source documents.
>
> The flow is: upload or URL goes into a Celery task, the task extracts text, splits it into semantically coherent chunks, embeds those chunks into a FAISS index and also builds a BM25 lexical index. When a question comes in, I run both indexes, fuse the two rankings with reciprocal rank fusion, rerank the candidates with a cross-encoder, drop chunks that overlap too heavily, compress the remaining chunks down to the query-relevant sentences, and hand that to whichever LLM provider the user configured. The answer comes back with the source filenames attached.
>
> The frontend is React, the backend is Django REST Framework with JWT auth, PostgreSQL for metadata, and uploaded files live in per-user directories on disk. It's a bring-your-own-key app, users paste their own Gemini, OpenAI, or Anthropic key, and that key is encrypted with Fernet before it hits the database.
>
> The piece I'm most proud of, and the piece I'd defend hardest in review, is the retrieval pipeline. The piece I'd change first is the index storage, and I'll explain why in a minute.

Follow-ups, answered short

- **Problem it solves.** Someone has a pile of documents and no way to get a trustworthy, cited answer out of them without reading all of it.
- **Most technically difficult part.** Not the model calls. It's that retrieval quality is a chain, and if the chunker puts a boundary in the wrong place, everything downstream inherits that mistake. So I made each stage independently testable and made each one degrade instead of crash.
- **What I'd change before many users.** Move the index out of process memory into a real vector store with per-user collections, and move files off local disk onto object storage.

2. Walk a document from upload to answer

> Upload hits `DocumentUploadView`. It validates the extension against a fixed allow list of `.txt`, `.md`, `.pdf`, `.docx`, sanitizes the filename down to its basename so nothing with a path separator survives, writes it into `documents/<user_id>/`, and creates a `Document` row plus a `Task` row with status pending.
>
> It returns immediately with the task id. `submit_task` hands the job to Celery over Redis. If the broker is unreachable it catches the error and runs the same job on a daemon thread inside

> the web process instead, so the feature still works locally.
>
> Inside the job, `load_documents` walks that user's directory, pulls text out per format. PDF goes through `pypdf` page by page, DOCX through `python-docx` paragraphs, text and markdown are read directly. Each file goes through `semantic_chunk`, and if that throws I fall back to splitting on blank lines. Chunks and their source filenames go into parallel lists. Then I build the FAISS index and the BM25 index over all the chunks, and progress gets written back to the task row.
>
> The client polls the task endpoint every couple of seconds for status, progress percentage, and message. That's how it knows ingestion finished.
>
> Then a question comes in, retrieval runs as I described, and the answer plus the chunks and source filenames get persisted on a `ChatMessage` row so you can reopen the conversation later and still see exactly what context produced the answer.

Follow-ups, answered short

- **Formats.** TXT, markdown, PDF, DOCX. Anything else is rejected at upload.
- **How the client knows it's done.** It polls the task id. There's no push channel for ingestion, only for the answer itself.
- **Extraction fails on one file.** The per-file loop has a try/except that skips that file and keeps going. If a file yields no text it's also skipped. One bad PDF doesn't kill the whole ingestion.
- **What would break in production.** The file is on local disk, so a second Gunicorn worker or a second host wouldn't see it. That's a real limitation.

3. The retrieval pipeline

> Five stages, and each one has a job.
>
> First, hybrid retrieval. I pull up to ten candidates from FAISS and ten from BM25, then fuse them with reciprocal rank fusion.
>
> Second, reranking. A cross-encoder, `ms-marco-MiniLM-L-6`, reads the query and each candidate chunk together and scores how well the chunk actually answers that query. That's much better than cosine similarity because the query vector and the chunk are compared as a pair. I keep the top three.
>
> Third, overlap removal. After reranking I may have three near-identical chunks that happen to sit either side of a chunk boundary. I compute Jaccard similarity on the word sets and drop anything above 0.6 overlap against something already kept. It costs no model call.
>
> Fourth, compression. Each surviving chunk gets split into sentences, each sentence is embedded, and I keep only sentences with cosine similarity above 0.3 against the query. If a chunk compresses to nothing, I keep its single best sentence rather than dropping the chunk entirely.

> Fifth, generation. The compressed chunks, the chat history, and a prompt telling the model to answer only from context go out to the provider.
>
> Every stage except the provider call has a fallback. If compression throws, I use the reranked chunks. If index building throws, the index is None and chat returns a clear 400 rather than crashing.

Follow-ups, answered short

- **Why ten then three.** Retrieval is cheap and recall-oriented, reranking is expensive and precision-oriented. Pull wide, then cut hard.
- **What RRF contributes.** FAISS gives me distances and BM25 gives me scores. Those two numbers are not comparable. RRF only looks at rank position, so it fuses them without needing a shared scale or tuned weights.
- **If compression fails.** Caught, and I send the reranked chunks instead.
- **Honest weakness.** The non-streaming chat endpoint only does the fusion step with `top_k=3` and skips rerank, overlap removal, and compression entirely. The streaming endpoint does the full chain. That inconsistency is a bug I'd fix first, by extracting one shared retrieval service both views call.

4. Why dense search and BM25 together

> They fail in opposite directions, which is exactly why you want both.
>
> Dense retrieval matches meaning. If someone asks "how do I stop it from running twice", the embedding will find a chunk that says "idempotency keys prevent duplicate execution", because those two sentences share almost no keywords. But dense retrieval is bad at exact tokens. Ask about `SECURE_HSTS_SECONDS` or a specific error code, and an embedding will happily return a topically similar chunk that does not contain that literal string.
>
> BM25 is the reverse. It's extremely good at exact strings, and it's bad at paraphrase. It has no idea that "sign out" and "log out" mean the same thing.
>
> The fusion matters as much as having both. I can't just add FAISS distances to BM25 scores, because one is a distance in embedding space and the other is an unbounded relevance score. Reciprocal rank fusion sidesteps that entirely. For each list I add `1 / (K + rank)` with K at 60, and sum. K being large flattens the curve so that a document ranked first in one list doesn't completely bury a document ranked first in the other.
>
> The classic case where this pays off is a question containing both a concept and an identifier, like "what does our HSTS setting do and where is it configured". BM25 finds the settings file chunk. Dense finds the chunk explaining what HSTS does. Fused, you get both.

Follow-ups, answered short

- **BM25 tokenization.** Whitespace split on a lowercased string. Simple, and a weakness, because `SECURE_HSTS_SECONDS` and `secure hsts seconds` become different tokens. A real tokenizer with stemming would be an upgrade.
- **RRF's K.** It controls how quickly a top rank decays into irrelevance. Small K makes rank one dominate. Large K flattens things so agreement across the two lists matters more than a single strong placement.
- **How I'd prove hybrid beats dense-only.** Hold the retrieval code fixed and swap `hybrid_search` for plain `search`, then measure answer faithfulness and context relevancy on the same eval dataset. The project already has DeepEval wired up for exactly that comparison.

5. Semantic chunking

> Fixed-size chunking with overlap is the default and it's fine, but it cuts mid-thought. You get a chunk that ends halfway through an explanation and a next chunk that starts with the answer.
>
> So I embed every sentence first, then walk the sentences in order comparing each one to the one before it with cosine similarity. If the similarity drops below 0.45, that means the topic changed, so I cut a boundary there. If adding the next sentence would push the chunk past 1500 characters, I cut on size instead.
>
> Sentence splitting is a regex that breaks after `.`, `?`, or `!` followed by whitespace, then I rejoin anything that got split after a known abbreviation like "Dr." or "e.g.". It's not perfect, but it's predictable.
>
> Two guards. If there are two or fewer sentences in the whole document, I return it as a single chunk rather than doing pointless math. And if `semantic_chunk` raises anywhere in the loader, I fall back to splitting on blank lines, so a bad embedding never means zero documents indexed.

Follow-ups, answered short

- **Effect of chunk size.** Bigger chunks give more context but dilute embeddings and waste prompt tokens. Smaller chunks are precise but lose context. 1500 characters is roughly 350 to 400 tokens, which is a reasonable balance.
- **Very long sentences.** No special handling. A 5000-character sentence becomes its own chunk and likely exceeds the soft limit on its own. A character-based tokenizer with sentence-aware fallback would handle this better.
- **Semantic versus fixed.** I'd benchmark both on the eval dataset. My hypothesis is semantic wins on prose like articles and papers, and loses on reference material like API docs where the blank-line structure is already meaningful and cheap.

6. How documents are isolated between users

> Two separate mechanisms, and one of them is a real weakness.
>
> The sound one is storage and queries. Files go into `documents/<user_id>/`, and every document and task query is filtered by `user=request.user`. Filenames are reduced with `Path(name).name`, so a traversal attempt collapses to the basename before it ever touches the filesystem. So even at the data layer, user A cannot address user B's file.
>
> The unsound part is retrieval state. The chunk list, the FAISS index, the embeddings array, and the BM25 index are module-level globals. `ensure_documents_loaded` compares the current global user id to the request user and rebuilds the whole index if they differ. That means one process serves one user's documents at a time, and it rebuilds from scratch every time the user changes.
>
> That breaks in two situations. If two requests for different users hit the same worker concurrently, whichever finishes last wins and the other user gets the wrong index. And `load_documents` mutates the global `docs` list in place with `clear()`, so an in-flight search can see a half-rebuilt list.
>
> With Gunicorn and multiple workers it gets stranger, because each worker has its own copy of these globals and they'd disagree about whose documents are loaded.
>
> I'd fix it by keying the index cache on user id, so a dict from user to its own index set, and by never mutating a live list in place. Beyond that, I'd push the index into a real store, either Qdrant, pgvector on the Postgres I already run, or Redis, with per-user collections. Then there's one shared index, it's consistent across workers, and there's no per-request rebuild.

Follow-ups, answered short

- **Can two requests overlap during a rebuild.** Yes, and that's the actual bug.
- **State shared across workers.** The embedding model and reranker should be per-process, that's fine and desirable, they load once. The index state is the problem, because each worker builds its own independently.
- **Redesign for multi-user.** Per-user index collections in a shared vector database, plus a per-user cache key and immutable index snapshots in the app layer.

7. Background tasks

> Every async operation has a row in the `Task` table with a UUID primary key. The status field is one of pending, processing, completed, failed, or cancelled, and there's a progress integer, a message, a JSON result, and an error string.
>
> `_run_task` is a single Celery task that all three job types go through. The view passes a dotted Python path to the real job function rather than the function itself, because Celery has to serialize arguments through JSON and a closure isn't serializable. Inside the task it imports

> the module, resolves the function, and calls it with two callbacks: `update` for progress and `is_cancelled`.
>
> The interesting part is cancellation. When someone hits cancel, the API just writes `status = cancelled` to the row. The running job doesn't get killed. Instead, the next time the job calls `update` or `is_cancelled`, it sees the flag, `is_cancelled` raises `TaskCancelled`, and the job unwinds through its normal checkpoints. The file loop in `load_documents` checks between files, and there's a check before index building.
>
> On failure, the exception string goes into the task's error field and the status becomes failed, and the API surfaces it.

Follow-ups, answered short

- **Redis unavailable.** `submit_task` catches the enqueue error and starts a daemon thread. Work still completes locally.
- **How cancellation reaches a running job.** Cooperative. Flag in the database, checked at checkpoints. Not a hard kill, which is the right trade for half-finished file writes.
- **Could a task run twice.** Yes. If the worker dies after doing the work but before writing completed, Celery's default redelivery would run it again. I'd make ingestion idempotent by keying on a content hash, so reprocessing the same file is a no-op, and by upserting the `Document` row on the existing unique constraint of user plus filename.

8. What happens when Celery can't enqueue

> `submit_task` wraps the `delay` call in a try/except. Any exception and it imports threading, spawns a daemon thread running the same `_run_task` function, and returns. The caller never learns which path was taken.
>
> What that buys you: the app still works on a laptop with no Redis, which is genuinely useful for local development and for a demo.
>
> What it costs: the thread dies with the web process, so any half-finished ingestion is lost with no retry and no durable queue. There's no backpressure, so a burst of uploads spawns a burst of threads competing for the same CPU and the same global index. And it's invisible, because nothing surfaces that the app is running degraded.
>
> Would I rather fail the request? For production, yes. I'd fail fast with a 503 and alert, because silently degrading is how you end up debugging a mystery. The fallback is fine in development and wrong in production, so it should be gated on an environment flag. A better middle ground is a durable fallback, for example writing the job to a database queue that a lightweight poller picks up, which gives you the no-Redis ergonomics without the volatility.

Follow-ups, answered short

- **Process exits.** The daemon thread dies with it. Task row stays in processing forever, because nothing sets it to failed.
- **How would you expose it.** A health check that reports broker connectivity, and a metric counting fallback executions.
- **Fail or degrade.** Gate it. Degrade in dev, fail loudly in prod.

9. LLM provider keys

> Users bring their own key. Provider, model, and the encrypted key all live on `UserProfile`.
>
> The encryption module wraps Fernet. `encrypt_value` produces ciphertext with an `enc:` prefix and stores that. `decrypt_value` checks for the prefix, and if it's there, strips it and decrypts. If it's not there, it assumes a legacy plaintext value and returns it unchanged, which is how I kept already-stored keys working through the migration.
>
> Behaviour on misconfiguration matters here. If no key is configured in development, `encrypt_value` returns plaintext so local work isn't blocked. If an encrypted value is found but no key is set, or the key is the wrong key, it logs an error and returns an empty string, which surfaces downstream as a provider auth failure instead of a confusing decrypt traceback.
>
> Settings refuse to boot in production without a valid Fernet key. In development it validates the format if one is present, but doesn't require it.

Follow-ups, answered short

- **Key rotation.** Not solved. Fernet supports a key list via `MultiFernet`, so decrypt tries each key and encrypt writes with the current one. I'd use that and re-encrypt in the background on next write.
- **Wrong key.** `InvalidToken`, caught, logged, empty string returned.
- **What must never be logged.** The key itself, the decrypted value, and any Authorization header. My middleware logs method, path, status, duration, user, and a request id, none of which touch credentials.

10. Authentication in the browser

> Short-lived JWT access token, long-lived refresh token in an HttpOnly cookie.
>
> Signup and signin return an access token in the body. The refresh token goes into an HttpOnly cookie, which means JavaScript can't read it, so XSS can't steal it. The frontend keeps the access token in memory and sends it as a Bearer header.

> Access tokens last 30 minutes, refresh tokens one day. When the access token expires, the API client catches the 401 and calls the refresh endpoint. That endpoint reads the cookie, validates the refresh token, returns a new access token, and rotates the cookie.
>
> Logout is a blacklist. The refresh token's jti gets added to the blacklist table and the cookie is cleared, so that token can't be used to mint another access token even if someone held onto it.
>
> The signing key comes from `JWT_SECRET_KEY`, falling back to Django's secret key if unset. In production settings also require the secret key to be present rather than silently using a development default.

Follow-ups, answered short

- **Why HttpOnly for refresh.** It's the long-lived credential, so it's the one worth taking out of reach of injected script. The access token is short enough that being readable is an acceptable trade for keeping it simple.
- **Production cookie flags.** Secure, HttpOnly, and SameSite, plus HSTS with a one-year max-age including subdomains and preload, SSL redirect, and nosniff, all switched on when `DEBUG` is false.
- **Both expire.** The user sees the login screen. There's no silent extension.

11. URL ingestion and SSRF

> URL ingestion takes a user-supplied string and makes the server fetch it, which is a textbook SSRF hole, so there's a validation step before anything hits the network.
>
> `_is_safe_user_url` parses the URL, requires the scheme to be `http` or `https`, requires a hostname, then calls `_is_private_or_local_host`. That resolves the hostname with `getaddrinfo` and checks every returned address against `is_private`, `is_loopback`, `is_link_local`, `is_multicast`, and `is_reserved`. Any one of them matching rejects the whole thing. And if DNS resolution fails, I reject, because a name I can't resolve is not a name I want to connect to.
>
> YouTube and Wikipedia are special-cased before this path, since they need transcript and article handling rather than generic HTML scraping.

Follow-ups, answered short

- **Which ranges.** Loopback, RFC 1918 private, link-local including the 169.254 metadata address, multicast, and reserved. Note that 0.0.0.0 and IPv6 loopback come along for the ride.
- **Can DNS change between check and connect.** Yes, that's a TOCTOU gap and the honest answer. The DNS rebinding window is real. The fix is to connect to the resolved IP directly while setting the Host header and doing TLS SNI properly, or to resolve and validate inside the same code path as the connect.

- **How I'd test it.** Unit tests with a mocked `getaddrinfo` covering `http://127.0.0.1`, `http://169.254.169.254/latest/meta-data/`, `http://[::1]`, `file:///etc/passwd`, and a `gopher://` scheme, plus an integration test with a local HTTP server to confirm the fetch is actually refused.

12. Streaming chat

> `ChatStreamView` returns a `StreamingHttpResponse` of server-sent events. Each event is `data:` plus a JSON object and a blank line.
>
> The heavy lifting finishes before the stream opens. Retrieval, reranking, overlap removal, compression, profile validation, and key decryption all happen synchronously in the `post` method. So if something's wrong, like an invalid model for the provider or no indexed documents, you get a real HTTP status code with a JSON error body. That's deliberate, because once the first byte is flushed you can only set the status by breaking the connection.
>
> After that the generator yields `{"token": "..."}` events, accumulating the full answer as it goes.
>
> Errors mid-stream come through the same channel as `{"error": "..."}`. ValueError, which is a config problem, and ProviderAPIError, which carries the provider's status code, both get yielded as error events and then the generator returns.
>
> When the iterator is exhausted, the final assembled answer is persisted to the `ChatMessage` row inside the generator's completion block. If the browser disconnects mid-stream, the generator is closed, the partial answer is not saved, and no orphaned message is created.

Follow-ups, answered short

- **Errors after the stream starts.** Delivered as an error event on the same channel. The status code is already committed by then.
- **Browser disconnects.** Generator closes, partial answer discarded, nothing written.
- **Persistence.** Once at the end, from the accumulated string, with the conversation, question, sources, and chunks.

13. Evaluating the RAG

> Two separate pieces, and separating them matters.
>
> There's an endpoint that generates evaluation questions from the user's own documents, and another that runs DeepEval metrics against a dataset and writes results to `EvaluationResult`. The metrics are faithfulness, answer relevancy, and context relevancy.
>
> Faithfulness is the hallucination check. It asks whether every claim in the answer is actually supported by the retrieved context, and it is the metric I'd put the most weight on, because a

> fluent answer that contradicts the source is worse than no answer.
>
> Answer relevancy asks whether the answer addresses the question that was asked, catching the failure where retrieval worked but the model answered something adjacent.
>
> Context relevancy scores the retrieved chunks themselves, so it measures the retrieval stage in isolation.
>
> There's also a feedback table. Thumbs up or down, plus an optional failure tag of bad retrieval, hallucination, incomplete, or other. The bad-retrieval tag is the useful one, because it separates "the chunks were wrong" from "the chunks were fine and the model still got it wrong", and those two problems have completely different fixes.
>
> The synthetic questions are the weakness. An LLM generating questions from the same documents that retrieval reads will tend to generate questions the corpus answers cleanly, which flatters the numbers. I'd want human-written questions for the headline numbers, with synthetic ones used only for volume.

Follow-ups, answered short

- **Using feedback.** Aggregate failure tags, pull the bad-retrieval examples, check whether the answer chunks contained the information. If they didn't, the chunker or the retrieval stage is the bug, not the prompt.
- **Hard queries.** There's an endpoint that surfaces low-scoring or thumbs-down answers so they can be promoted into the eval set, which closes the loop between production and evaluation.

14. What the deployment runs

> Compose brings up five things: PostgreSQL, Redis, the Django backend, a Celery worker, and the React frontend. There's a `render.yaml` that describes a Render web service plus a Postgres instance, and a GitHub Actions workflow that runs backend and frontend checks with a Postgres service container.
>
> Environment variables come from the process environment via `python-dotenv`, with `.env.example` documenting what's needed. Production requires `DJANGO_SECRET_KEY`, `JWT_SECRET_KEY`, `FIELD_ENCRYPTION_KEY`, `DJANGO_ALLOWED_HOSTS`, and `DJANGO_CORS_ALLOWED_ORIGINS`, and the settings module raises at import time if any are missing rather than booting with a weak default.
>
> In production, `DEBUG` false turns on SSL redirect, secure cookies, HSTS for a year including subdomains and preload, and nosniff. Static files go through WhiteNoise with a compressed manifest so there's no separate static host.
>
> What I would not claim is that this is running in production. It's configured for it.

Follow-ups, answered short

- **Where files live in production.** On the container's local filesystem, which means they vanish on redeploy and aren't shared between the web service and any scaled worker. This is the single biggest deployment gap, and the fix is S3 or equivalent plus a `Document.storage_key` column instead of assuming a path.
- **Worker restarts mid-ingestion.** The task row is stuck in processing. Nothing detects that. I'd add a heartbeat or a started-at threshold sweep that marks orphaned tasks failed so the user can retry.
- **Required env vars.** The five listed above, plus `DATABASE_URL` and `CELERY_BROKER_URL`.

15. What the tests establish

> The backend suite is around 37KB of Django tests, so it's substantial rather than token. It covers signup, signin, refresh, logout, and token rotation. It covers user scoping, meaning user A's document id returns 404 for user B and not someone else's file. It covers the task lifecycle, including cancel before start, cancel mid-run, and the failed path. It covers ingestion validation with bad extensions and unsafe URLs. It covers collections, search, and the rerank endpoint. It covers chunking and compression directly as unit tests, plus encryption round-trips and the legacy plaintext fallback, and the logging and middleware behaviour.
>
> What it does not establish is anything about concurrency or scale. There is no test that runs two users against the same in-memory index at the same time, and that is exactly where I believe the bug is. There's no test for index rebuild under load.
>
> The missing test I'd write first is a threaded test with two users hitting `ensure_documents_loaded` concurrently, asserting each gets their own chunks. It would fail today, which is the point.

Follow-ups, answered short

- **What's mocked.** External provider HTTP calls, the embedding and reranker models where they'd be slow, and network fetches for URL ingestion. Real Postgres runs in CI via the service container.
- **Frontend.** Create React App's default smoke test plus the build step.
- **Rule I'd follow.** Prefer end-to-end tests as the primary mechanism, because a mocked retrieval pipeline test proves your mock works, not your retriever.

16. Git history

> Two changes worth talking about.
>
> The first is hybrid search. Originally retrieval was dense only, FAISS plus rerank. BM25 and the RRF fusion were added on top of that rather than replacing dense search, which is the whole

> point of having both. The diff shows `hybrid_search` taking the dense results and the BM25 results and merging them by rank position, with the `K = 60` constant right there in the function.
>
> The second is SSE streaming. `ChatStreamView` was added alongside `ChatView`, and the interesting decision was that all the retrieval work happens before the stream opens, so failures still produce proper status codes.
>
> Worth being precise about state: the committed history shows the pinned-conversation migration as the latest change. The current working tree also has uncommitted work on top of that. So I'd say what's committed and what's in progress separately rather than blurring them.

Follow-ups, answered short

- **What changed in retrieval.** A third ranking source and a fusion step. Rank-based, not score-based, precisely because the scores aren't comparable.
- **How the frontend consumes it.** `apiClient` reads the response body as a stream, splits on the `\n\n` boundary, JSON-parses each `data:` payload, and appends `token` to the message already on screen. It also handles the 401-refresh-retry before the stream starts.
- **Uncommitted work.** Conversation pinning is in the working tree alongside the committed migration.

The hard ones

What's the weakest part of this architecture?

> The module-level index globals. Everything else degrades gracefully, but that one thing means the app is effectively single-user per process and has a correctness bug under concurrent multi-user load. Everything else is a scaling limit. That one is a correctness problem.

What assumption could cause this system to fail?

> That the index and the user's documents stay in agreement. The index is a derived cache of files on disk, rebuilt opportunistically. If a file changes out of band, or two workers rebuild at different times, the index can be stale relative to the disk and there's no version check. A content hash or a modified-time stamp on the index would fix it.

Which part of a RAG answer can be wrong even when retrieval is perfect?

> The generation step. Retrieval returning the right chunk only guarantees the right chunk was in the prompt. The model can still over-generalize, answer from its own prior knowledge instead of the context, or misread an ambiguous passage. That's what the faithfulness metric is for, and it's also why the prompt instructs it to say it doesn't know rather than guess.

What would break first under load?

> Index rebuilds. Every user switch triggers a full re-extract and re-embed of all their documents, synchronously, inside a request. That's the single slowest operation in the app and it's on the request path. The fix is incremental indexing, where adding one document only embeds the new chunks and updates the index, plus moving the rebuild behind the task queue instead of blocking the request.

Which security issue would you audit first?

> SSRF, because it's the one place a user-controlled string causes the server to make an outbound request, and because the current check has a genuine TOCTOU gap between the DNS validation and the connection. Second on the list is the encrypted-key rotation story.

If you could rewrite one component?

> `views.py`. It's 86KB and holds roughly 50 view classes plus document extraction, URL fetching, HTML parsing, mojibake repair, and formula normalization. The retrieval logic should live in one service that both the streaming and non-streaming views call, which would also have eliminated the inconsistency I mentioned earlier. That's the change with the best ratio of risk removed to work done.

What would change at 100x users?

> Index storage, retrieval caching, and ingestion throughput. Not the API layer, which is stateless and fine. Not the database, which is doing nothing demanding. The whole system's weight sits on the retrieval path, and that's where I'd spend the redesign.